

# YAQA: 모델 출력을 보존하는 적응형 반올림

Model-Preserving Adaptive Rounding

---

Albert Tseng, Zhaofeng Sun, Christopher De Sa (2026)

quantizer는 그대로 두고, full-model KL Hessian의 Kronecker 근사로 rounding 결정을 바꾼다

# LLM 배포에서 양자화가 맡는 일

고정밀도 weight를 적은 bit로 표현해 memory traffic과 지원 hardware의 계산량을 줄인다

01 원본 모델 수십억 개 고정밀도 weight



02 low-bit 표현 weight마다 representable point 선택



03 압축 모델 작은 storage와 memory bandwidth



04 보존 목표 출력 분포가 원본과 가까워야 함

압축률만 맞추는 것이 아니라 어떤 rounding이 원본 model behavior를 덜 바꾸는지가 문제다

# quantizer와 rounding algorithm은 다르다

표현 가능한 값의 집합과 그 집합으로 보내는 결정은 분리된 구성요소다

| 구성요소      | 정하는 것                     | 예시                    | inference에 미치는 영향         |
|-----------|---------------------------|-----------------------|---------------------------|
| quantizer | representable point와 code | E8P, QTIP, INT4       | datatype·decode·kernel 형식 |
| rounding  | 각 weight가 받을 point        | LDLQ, YAQA            | 같은 format 안의 code 선택      |
| YAQA의 위치  | rounding만 교체              | quantizer-independent | 추가 format overhead 없음     |
| 남는 비용     | Hessian 추정과 반복            | offline calibration   | 추론 전 준비 비용은 증가            |

YAQA의 결과는 선택한 quantizer가 표현할 수 있는 값 밖으로 나가지 않는다

# 목표는 원본 출력 분포와의 KL을 줄이는 것

입력 X마다 원본 M과 양자화 모델 Mq의 전체 output distribution을 비교한다

01    입력 X                      data distribution에서 sample

---



02    원본 M                      output distribution 생성

---



03    양자화 Mq                      같은 입력의 output 생성

---



04    KL objective                      기댓값이 작은 quantized weight 선택

---

model-preserving은 정답 label만 맞추는 것이 아니라 전체 출력 분포를 가깝게 두는 목표다

# immediate activation error는 왜 proxy인가

한 layer 출력을 바로 맞추는 것과 이후 모든 layer를 지난 model output을 맞추는 것은 다르다

| 관점         | 측정 대상                  | 장점                      | 놓치는 것               |
|------------|------------------------|-------------------------|---------------------|
| LDLQ/GPTQ  | 현재 layer의 output error | forward input으로 계산 가능   | 후속 layer의 증폭·상쇄     |
| YAQA 목표    | full-model output KL   | model behavior와 직접 연결   | 정확 계산은 너무 큼         |
| Hessian 의미 | rounding error의 민감 방향  | second-order 중요도 제공     | 근사의 질에 의존           |
| 핵심 차이      | H1 대 full H            | tractability 대 fidelity | 둘은 같은 objective가 아님 |

local error가 작아도 후속 layers를 거친 최종 분포가 더 가까워진다고 보장되지 않는다

# full Hessian은 그대로 저장할 수 없다

m by n weight matrix를 펼치면 민감도 행렬 H는 mn by mn이 된다

| object       | shape           | 역할                        | scale 문제             |
|--------------|-----------------|---------------------------|----------------------|
| weight W     | m by n          | 한 linear layer            | mn parameters        |
| local H1     | n by n          | input-channel sensitivity | LDLQ가 저장 가능          |
| true H       | mn by mn        | full-model KL curvature   | 직접 연산·저장 불가          |
| YAQA factors | m by m + n by n | output·input 구조 근사        | $O(m^2+n^2)$ storage |

YAQA의 핵심 압축은 weight가 아니라 Hessian을 두 작은 factor로 표현하는 데서 시작한다

# LDLQ는 input channel 방향으로 error를 넘긴다

한 번의 update를 왼쪽에서 오른쪽으로 펼치면 input-side feedback만 남는다



---

$L^T$ 는 H1의 LDL factor에서 오며, 이미 생긴 error가 뒤 input-channel 결정을 바꾸게 한다

# SND는 rounding dependency의 깊이이다

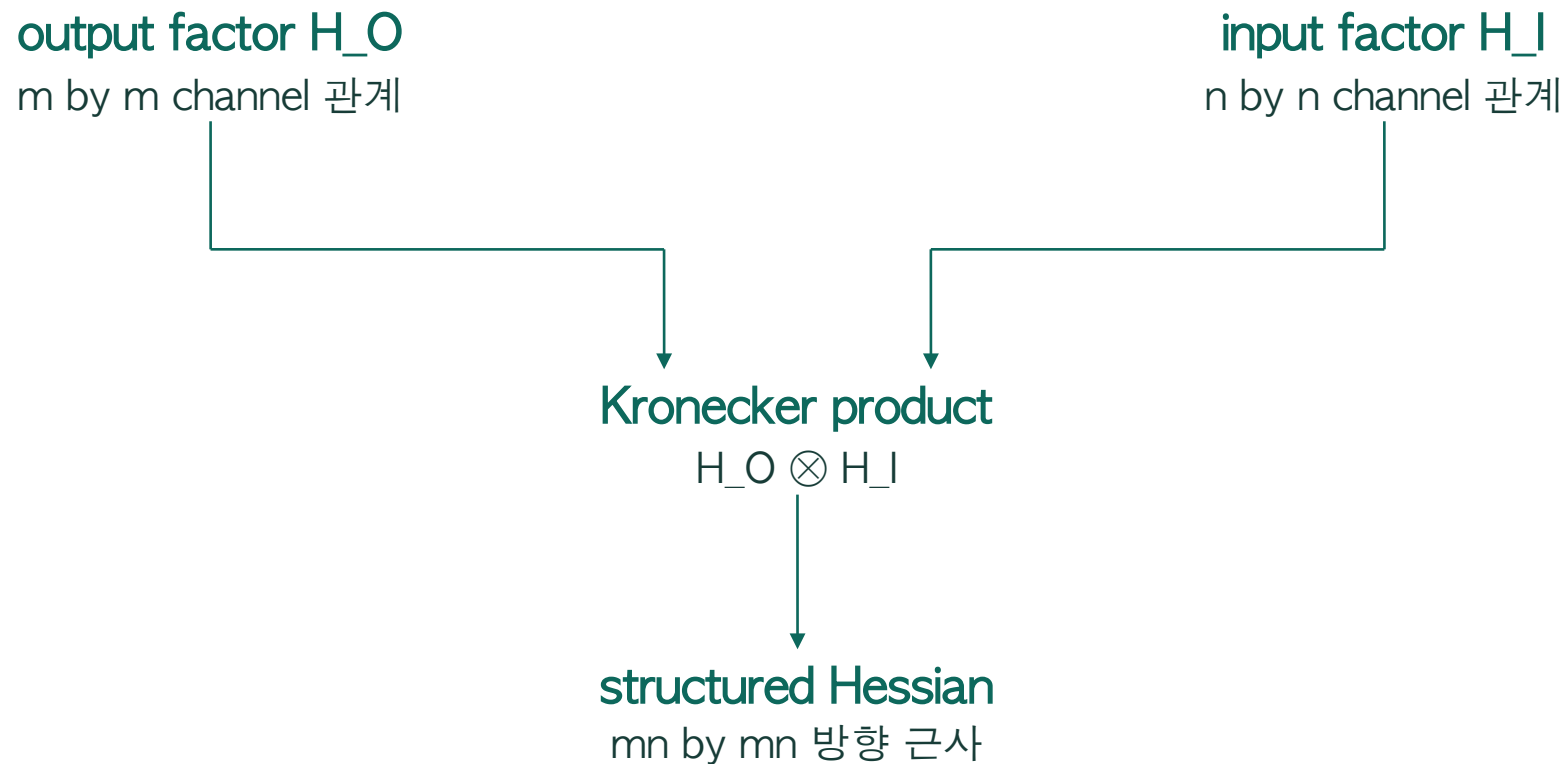
support graph에서 한 weight의 결정이 다음 weight에 영향을 주는 longest chain을 센다

| 구조            | dependency           | SND 해석     | iteration bound |
|---------------|----------------------|------------|-----------------|
| diagonal      | 서로 독립                | 깊이 1       | 한 번에 round      |
| input LDL     | input channel 순서     | 최대 $n$     | LDLQ 약 $n$ 단계   |
| dense full H  | $mn$ coordinates 연결  | 최대 $mn$    | 직접 방식은 비현실적     |
| Kronecker LDL | output + input chain | 최대 $m+n-1$ | YAQA 약 $m+n$ 단계 |

Kronecker 구조는 full dependence를 두 축의 chain 길이 합으로 제한한다

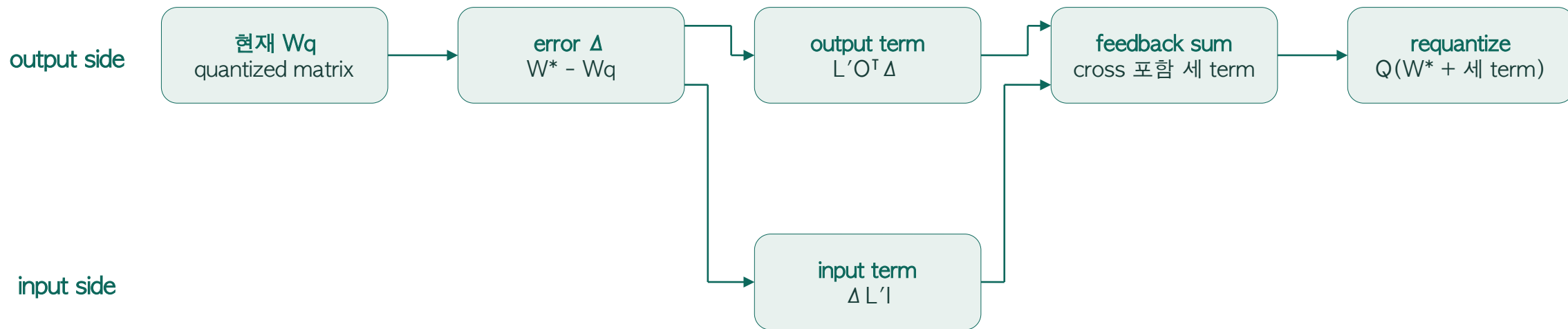
# Kronecker factor가 두 channel 축을 결합한다

두 작은 민감도 행렬을 곱 구조로 묶어 full Hessian의 주요 방향을 근사한다



# YAQA는 output과 input 양쪽 error를 전달한다

현재 rounding error에서 세 feedback term을 만들고 원본 weight에 더해 다시 quantize한다



LDLQ의  $\Delta L'I$ 에  $L'O^T \Delta$  와  $L'O^T \Delta L'I$ 가 추가되어 output-channel 관계도 rounding에 반영된다

# 세 feedback term을 따로 읽기

$L'O$ 와  $L'I$ 는 각각  $H_O$ 와  $H_I$ 의 block LDL decomposition에서 얻은 strictly lower part다

| term                  | 전파 축           | 직관                        | LDLQ에 존재 |
|-----------------------|----------------|---------------------------|----------|
| $\Delta L'I$          | input          | 이전 input-channel error 전달 | 예        |
| $L'O^T \Delta$        | output         | output-channel error 전달   | 아니오      |
| $L'O^T \Delta L'I$    | output + input | 두 축의 결합 의존성               | 아니오      |
| $Q(W^* + \text{sum})$ | codebook       | 세 correction 뒤 다시 선택      | 같은 Q 사용  |

dual feedback은 두 개의 독립 보정만이 아니라 두 축이 만나는 cross term까지 포함한다

# 교육용 2 by 2 rounding trace의 설정

nearest-integer quantizer와 작은 LDL factors를 둔 재계산 가능한 예시이며 논문 실험값이 아니다

| 항목              | 값                                | 의미                             |
|-----------------|----------------------------------|--------------------------------|
| 원본 $W^*$        | $[[0.45, 1.55], [2.45, 2.55]]$   | 양자화 전 weight                   |
| 초기 $W_0$        | $[[0, 2], [2, 3]]$               | $W^*$ 를 nearest integer로 round |
| $L'_0$ 와 $L'_1$ | $[[0, 0], [0.5, 0]]$             | 두 축의 strictly lower factor     |
| 초기 $\Delta_0$   | $[[0.45, -0.45], [0.45, -0.45]]$ | $W^* - W_0$                    |

quantizer의 integer codebook은 고정하고 feedback이 선택 code만 바꾸는지 확인한다

# 첫 update에서 code 하나가 2에서 1로 바뀐다

$\Delta 0$ 에 output, input, cross factor를 적용해 새 quantization target을 만든다

| 계산          | 행렬 값                               | 결과                            |
|-------------|------------------------------------|-------------------------------|
| output term | $[[0.225, -0.225], [0, 0]]$        | $L'O^T \Delta 0$              |
| input term  | $[[-0.225, 0], [-0.225, 0]]$       | $\Delta 0 L'I$                |
| cross term  | $[[-0.1125, 0], [0, 0]]$           | $L'O^T \Delta 0 L'I$          |
| new target  | $[[0.3375, 1.325], [2.225, 2.55]]$ | $W^* + \text{세 term}$         |
| round $W1$  | $[[0, 1], [2, 3]]$                 | upper-right $2 \rightarrow 1$ |

같은 nearest-integer quantizer라도 feedback을 더한 target은 다른 code를 선택할 수 있다

# 두 번째 update 뒤 fixed point에 도달한다

새 code로 error를 다시 계산하고 같은 update를 반복해 code가 더 바뀌는지 확인한다

| 상태             | error 또는 target                                  | quantized matrix              |
|----------------|--------------------------------------------------|-------------------------------|
| W1 이후 error    | $\Delta 1 = \{([0.45, 0.55]), ([0.45, -0.45])\}$ | $W1 = \{([0, 1]), ([2, 3])\}$ |
| 두 번째 target    | $\{([0.8375, 1.325]), ([2.225, 2.55])\}$         | $W2 = \{([1, 1]), ([2, 3])\}$ |
| 세 번째 반복        | 같은 target과 code                                  | $W3 = W2$                     |
| stop condition | code가 더 바뀌지 않음                                   | fixed point                   |

---

Algorithm 1은 objective gradient가 0이 될 때가 아니라 quantized code가 안정되면 멈춘다

---

# Theorem 3.4가 보장하는 범위

정리는 가정 아래 error bound를 주며, 방향 유사도와 incoherence가 좋을수록 bound가 작아진다

| 요소             | bound에서의 역할                   | 가정·bound 해석                           | 말하지 않는 것               |
|----------------|-------------------------------|---------------------------------------|------------------------|
| cosine c       | H와 $H_O \otimes H_I$ mismatch | tightness: $c \rightarrow 1$ 이면 감소    | H를 정확히 복원              |
| incoherence    | trace term 제어                 | tightness: 낮을수록 유리                    | 모든 layer 동일            |
| rounding       | rounding error bound          | nearest/stochastic; $E\{e^2\} \leq B$ | 임의 quantizer           |
| rank condition | LDLQ bound와 비교                | LDLQ 비교: 낮은 output rank               | 항상 lower loss          |
| 결론             | error upper bound             | 가정: true KL H, PD factors             | downstream accuracy 보장 |

‘provably better’는 조건부 bound 비교이며 모든 설정의 품질 우위를 뜻하지 않는다

# true KL Hessian을 real Fisher로 관측한다

원본 model output에서 token을 sample하고 sequence gradient의 outer product를 평균한다

01 원본 logits                      model output distribution

---



02 Monte Carlo token              그 distribution에서 sample

---



03 sequence gradient  $G$               layer weight에 대한 gradient

---



04 real Fisher                      gradient outer product 기대값

---



05 factor sketch                       $H_O$ 와  $H_I$ 를 근사

---

training label이 아니라 original model distribution에서 얻은 real Fisher를 쓴다

# Sketch A: token 독립 가정으로 variance를 낮춘다

layer input  $x$ 와 output-error gradient를 token pair로 모아 factor power iteration을 수행한다

|    |                     |                             |
|----|---------------------|-----------------------------|
| 01 | input $x$           | 각 token의 layer input        |
| ↓  |                     |                             |
| 02 | error gradient      | 같은 sample의 output signal    |
| ↓  |                     |                             |
| 03 | token-level moments | independence 가정으로 평균        |
| ↓  |                     |                             |
| 04 | power iteration     | H1과 I에서 최대 3회               |
| ↓  |                     |                             |
| 05 | $H_I, H_O$          | biased low-variance factors |

더 많은 token pair를 쓰는 대신 sequence correlation을 무시해 bias가 생긴다

# Sketch B: sequence gradient로 factor를 구한다

identity에서 시작한 one-step power iteration을 per-sequence real Fisher gradient로 계산한다

01 sequence sample      원본 output에서 token sample

---



02 gradient matrix G      한 sequence의 weight gradient

---



03 input factor       $H_I = E[G^T G] / m$

---



04 output factor       $H_O = E[G G^T] / n$

---



05 single pass      같은 backward에서 둘 다 누적

---

token independence bias를 피하지만 독립 단위가 sequence라 충분한 data가 더 많이 필요하다

# Sketch A와 B의 tradeoff

둘 다 local H1보다 full Hessian 방향에 가깝지만 data regime에 따라 상대 우위가 달라진다

| 항목          | Sketch A               | Sketch B             | 해석               |
|-------------|------------------------|----------------------|------------------|
| sample unit | token pair             | sequence gradient    | B가 dependency 보존 |
| bias        | token independence로 존재 | 낮음                   | A의 구조적 근사        |
| variance    | 많은 token으로 낮음          | sequence 수에 의존       | B는 data 요구 큼     |
| main cost   | 20M tokens, 약 20 GPU-h | 64K × 2K, 약 30 GPU-h | 10B 기준 보고        |
| 관찰          | data-limited 때 우세 가능   | 충분한 data에서 대체로 강함    | 항상 B는 아님         |

A는 lower variance, B는 lower bias 쪽의 선택이며 둘 다 한계가 있다

# incoherence processing은 bound의 trace를 낮춘다

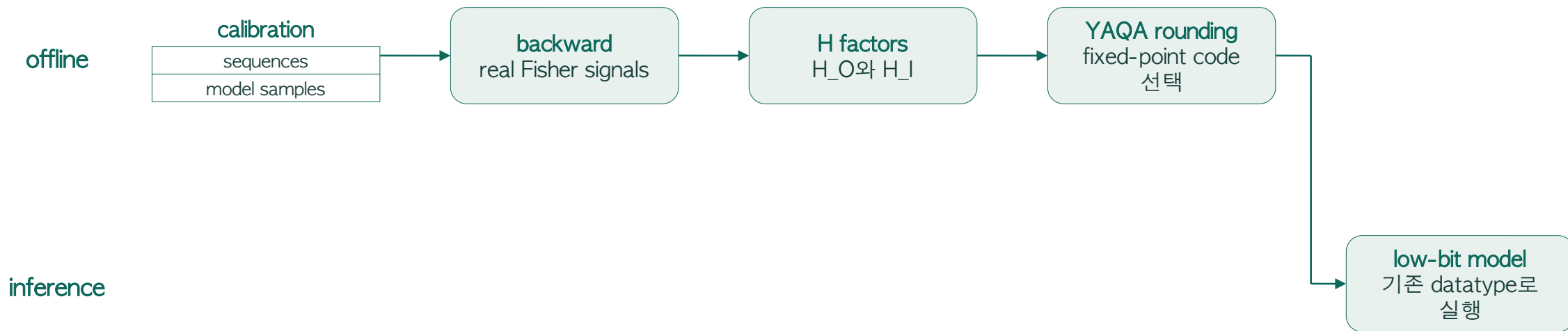
randomized Hadamard transform이 민감 방향을 좌표에 고르게 퍼뜨리도록 weight와 factors를 함께 바꾼다

| 단계               | 대상                     | 목적                 | 경계                 |
|------------------|------------------------|--------------------|--------------------|
| RHT              | weight channel basis   | 큰 coordinate 집중 완화 | 정확한 uniformity 아님  |
| factor transform | H_O와 H_I               | 같은 basis에서 LDL 수행  | factor도 같이 변환      |
| trace ratio      | processed 대 original D | 1보다 작으면 bound 감소   | empirical CDF      |
| experiment       | QTIP main setting      | LDLQ와 YAQA 모두 적용   | rounding 효과와 혼동 금지 |

같은 rotation 조건에서 YAQA가 더 나은 Hessian feedback을 쓰는 비교다

# offline sketch부터 inference까지의 경계

추론 전에 factors와 codes를 만들고, 배포 시에는 선택한 quantizer의 기존 표현만 사용한다



no inference overhead는 final format에 대한 말이며 offline 비용은 남는다

# main experiment를 읽는 조건

model, quantizer, finetuning, calibration, evaluation length를 함께 고정해야 숫자를 비교할 수 있다

| 항목             | reported condition                    | 비교 시 주의                 |
|----------------|---------------------------------------|-------------------------|
| models         | Llama 3.1/3.2, Gemma 3; Qwen appendix | model size별 분리          |
| main quantizer | QTIP 2, 3, 4 bit + incoherence        | same quantizer 비교       |
| finetuning     | main은 없음; recovery는 별도                | Table 1과 4 분리           |
| Sketch A       | 8K context, 20M tokens, 3 iterations  | B와 data 단위 다름           |
| Sketch B       | 2K context, 64K sequences             | KL/PPL eval length 8192 |

YAQA-A와 B는 rounding은 같고 Hessian sketch가 다르다

# Llama 3.1 8B에서 2-bit rounding 비교

QTIP, incoherence processing, no finetuning 조건의 Table 1 한 구간이다

| rounding    | KL W2 ↓ | PPL W2 ↓ | PPL C4 ↓ | 0-shot avg ↑ |
|-------------|---------|----------|----------|--------------|
| LDLQ        | 0.356   | 9.39     | 10.70    | 62.51        |
| GuidedQuant | 0.317   | 9.03     | 10.42    | 63.12        |
| YAQA-A      | 0.284   | 8.79     | 10.09    | 64.06        |
| YAQA-B      | 0.241   | 8.39     | 9.83     | 64.32        |

이 row에서는 YAQA-B가 KL과 두 perplexity를 가장 낮추고 평균 zero-shot도 가장 높다

# standard INT4에서도 rounding만 바꿀 수 있다

Llama 3.1 8B Instruct, incoherence processing, no finetuning의 Table 2 결과다

| method    | KL W2 ↓ | PPL W2 ↓ | PPL C4 ↓ | 0-shot avg ↑ |
|-----------|---------|----------|----------|--------------|
| BF16      | 0       | 6.50     | 8.02     | 69.82        |
| LDLQ      | 0.038   | 6.76     | 8.26     | 67.99        |
| DiscQuant | 0.061   | 6.83     | 8.37     | 67.68        |
| YAQA-A    | 0.028   | 6.71     | 8.21     | 69.11        |
| YAQA-B    | 0.029   | 6.72     | 8.23     | 68.92        |

reported INT4에서도 YAQA가 local baselines보다 낮은 KL을 보였다

# QAT 비교에서는 bit와 data 차이를 함께 본다

Gemma 3 12B Instruct에서 original-output KL은 YAQA가 낮지만 조건은 같지 않다

| method     | type | bits | KL W2 ↓ | data condition                      |
|------------|------|------|---------|-------------------------------------|
| Google QAT | QAT  | 4.5  | 0.089   | authors estimate 약 $100\times$ data |
| YAQA-A     | PTQ  | 4    | 0.058   | 20M-token sketch                    |
| YAQA-B     | PTQ  | 4    | 0.056   | 64K $\times$ 2K sequences           |
| original   | BF16 | 16   | 0       | reference distribution              |

낮은 KL은 original 보존이며 QAT의 보편적 품질 열세를 뜻하지 않는다

# recovery finetuning 뒤에도 차이는 남지만 줄어든다

QTIP과 incoherence processing을 쓴 Llama 3.1 8B의 Table 4 KL 비교다

| bits  | LDLQ KL             | YAQA-A KL      | YAQA-B KL  | best reduction |
|-------|---------------------|----------------|------------|----------------|
| 2     | 0.185               | 0.163          | 0.147      | 약 21%          |
| 3     | 0.048               | 0.042          | 0.038      | 약 21%          |
| 4     | 0.016               | 0.014          | 0.012      | 25%            |
| scope | same recovery setup | same quantizer | same table | rounded values |

recovery finetuning 뒤에도 YAQA의 KL 이점은 남았다

# Sketch B는 sequence 수와 KL 사이의 곡선을 만든다

Qwen3 8B, QTIP 2-bit, incoherence processing, no finetuning의 Table 9다

| method | sequences | GPU-h | KL W2 ↓ | PPL W2 ↓ |
|--------|-----------|-------|---------|----------|
| LDLQ   | 4K        | 1.5   | 0.285   | 10.81    |
| YAQA-B | 2K        | 1     | 0.227   | 10.33    |
| YAQA-B | 8K        | 4     | 0.212   | 10.34    |
| YAQA-B | 32K       | 15    | 0.206   | 10.20    |
| YAQA-B | 64K       | 30    | 0.205   | 10.16    |

2K에서도 reported gain이 있지만 더 많은 data의 개선 폭은 점차 작아진다

# 정확도 이득과 함께 지불하는 비용

추론 format은 그대로지만 offline calibration과 factor memory는 LDLQ보다 커진다

| 항목             | YAQA                   | LDLQ 대비                        | 조건·한계               |
|----------------|------------------------|--------------------------------|---------------------|
| factor storage | $O(m^2+n^2)$           | 대략 $2 \times$ Hessian storage  | lower triangles만 저장 |
| accumulation   | FP32 + regularization  | backward 필요                    | TF32는 불안정 보고        |
| rounding       | output-input matmul 반복 | 약 $2 \times$ quantization time | authors setup에서 작음  |
| Sketch A/B     | 약 20 / 30 GPU-h        | 추가 offline cost                | 10B reported setup  |
| inference      | same quantizer format  | YAQA 추가 overhead 없음            | kernel 성능은 Q에 의존    |

배포 runtime을 늘리지 않는 대신 quantization pipeline의 준비 단계가 더 무거워진다

# 논문 결과를 넘어서 말하면 안 되는 것

method의 강점은 clear하지만 approximation, data, comparison scope가 남는다

| 확인된 결과                       | 해석 경계                      | 남은 질문                          |
|------------------------------|----------------------------|--------------------------------|
| reported settings에서 lower KL | 모든 quantizer-model의 보장 아님  | 다른 architectures 재현            |
| Theorem 3.4 error bound      | 조건부 second-order bound     | bound와 actual KL 간격            |
| A/B가 H1보다 방향상 근접             | full H를 직접 계산하지 않음         | data-limited reliability       |
| no inference overhead        | offline GPU-memory cost 존재 | large deployment pipeline cost |
| 선택 baselines보다 우수            | omitted PTQ 전체와 비교 아님      | broader matched evaluation     |

YAQA는 강한 근거를 주지만 exact optimization이나 universal dominance는 아니다

# YAQA를 한 번에 정리하면

codebook을 바꾸지 않고 full-model sensitivity에 가까운 방향으로 rounding assignment를 바꾼다

- Quantizer는 값 집합을 정하고, YAQA는 그중 선택할 code를 정한다.
- LDLQ의 local proxy 대신 full-model KL Hessian의 Kronecker sketch를 쓴다.
- Input feedback에 output과 cross feedback을 더해 두 축의 error를 반영한다.
- A는 token-level lower variance, B는 sequence-level lower bias를 지향한다.
- 결과는 lower KL을 보이지만 theorem·bit·data·finetuning 조건이 붙는다.
- Inference format은 유지되지만 offline collection과 storage cost는 증가한다.

---

핵심은 같은 값 집합에서 model을 덜 바꾸는 code를 고르는 것이다